VERIQTA

Learn. Build. Operate. Scale. Production-Ready.

Production Pipelines. Reliability. Automation. At Scale.

AZURE DEVOPS PRODUCTION HANDBOOK

VOLUME 1

PAGES 1 - 20

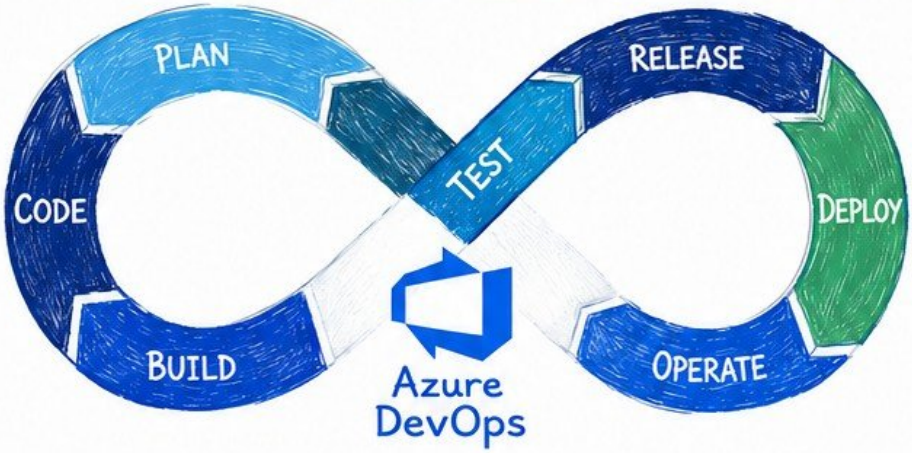
PLAN Strategy & Work

CODE Version Control

BUILD CI & Automation

TEST Quality Assurance

A Practical, Real-World Guide to Building and Managing Production-Grade DevOps with Azure



RELEASE Continuous Delivery

DEPLOY Infrastructure as Code

OPERATE Monitor & Run

SECURE Security Everywhere

REAL-WORLD SCENARIOS From the Field

PRODUCTION FOCUS Not Just Theory

BEST PRACTICES Proven at Scale

AUTOMATION FIRST Faster. Safer. Smarter.

BUILT FOR ENGINEERS By Engineers

VERIQTA

1. Introduction to Azure DevOps

Azure DevOps Production Handbook
Volume 1
Page 1 of 20

Azure DevOps is a cloud platform that helps teams plan, build, test, release, and monitor software — faster, safer, and at scale.

1 What is Azure DevOps?

Azure DevOps is a suite of cloud services from Microsoft that enables organizations to collaborate, automate, and deliver high-quality software continuously.

It provides end-to-end tools for the entire DevOps lifecycle in one platform.

2 Azure DevOps Services

- **Boards** — Plan, track and discuss work.
- **Repos** — Git repos, version control, code management.
- **Pipelines** — CI/CD automations for any platform.
- **Test Plans** — Test planning, manual & exploratory tests.
- **Artifacts** — Package feeds, dependencies, and artifacts.

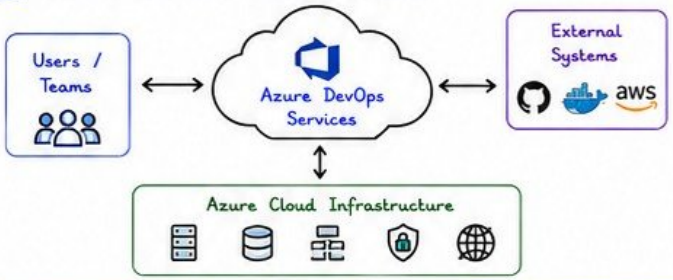
Integrated, secure, and built for enterprise scale.

3 DevOps Lifecycle

Plan → Code → Build → Test → Release → Operate

Azure DevOps supports the full DevOps lifecycle with automation, traceability and visibility.

4 Azure DevOps Architecture



5 Why Enterprises Use Azure DevOps

- End-to-end visibility and traceability
- Scalable and secure cloud platform
- Automated CI/CD for faster delivery
- Strong integrations with Microsoft ecosystem
- Compliance, governance and security at scale
- Supports hybrid and multi-cloud environments

6 Azure DevOps vs GitHub

	Azure DevOps	GitHub
Owned By	Microsoft	Microsoft
Best For	Enterprise DevOps	Developers & Open Source
Services	Boards, Pipelines, Test Plans, Artifacts, Repos & more	Code hosting, Actions, Packages, Issues
CI/CD	Built-in Pipelines	GitHub Actions
Integration	Deep Azure & Microsoft ecosystem	Wide open source ecosystem
Enterprise Ready	Yes (RBAC, Compliance, Audits, Policies)	Yes (Enterprise plans available)

Both are powerful. Azure DevOps offers more end-to-end capabilities for enterprise delivery.

7 Production Use Cases

- Continuous Integration & Delivery (CI/CD)
- Infrastructure as Code (IaC) deployments
- Multi-stage release management
- Compliance & audit automation
- Secure secret management
- Monitoring & feedback loops
- Large scale enterprise applications

Production Focus

- Reliability
- Security
- Scalability
- Compliance
- Automation

8 Best Practices

- Use Git branching strategy (main, develop, feature)
- Protect critical branches and enforce PR reviews
- Automate builds, tests and quality gates
- Use variable groups and secure variable handling
- Enable approvals and checks for releases
- Monitor pipeline health and failures
- Keep libraries, tasks and agents up to date
- Document, standardize and reuse templates

Remember:
Consistency + Automation + Visibility = Reliable Software Delivery

Azure DevOps brings people, processes, and technology together to build better software — faster.
Master it. Automate it. Scale it. Deliver it.

VERIQTA

2. Azure DevOps Organizations and Projects

Organizations and Projects are the foundation of Azure DevOps.
Design them right for security, scalability, and long-term success.

1 Organizations

An organization is the top-level container in Azure DevOps.
It represents your company or business unit.

- ☑ Unique DNS name (e.g., contoso)
- ☑ Global namespace
- ☑ Contains one or more projects
- ☑ Shared billing and settings
- ☑ Central identity and access control

Choose a simple, permanent name.
Changing it later is not possible.

2 Projects

Projects are containers for your development work.
They isolate data, settings, and security.

- ☑ Isolated security boundaries
- ☑ Separate pipelines, repos, boards, and settings
- ☑ Custom process and visibility
- ☑ Easier delegation and scaling

Create one project per product, application, or major service.

3 Users and Teams

Manage who can access and what they can do.

Users

- Individual identities
- Can belong to multiple teams
- Licensed via Microsoft Entra ID

Teams

- Security groups
- Simplify permission management
- Can be nested

Always assign permissions to teams, not individual users.
It scales and is easier to manage.

4 Permissions

Azure DevOps uses role-based access control (RBAC).

Common Built-in Roles

- **Project Administrator** — Full control over the project
- **Build Administrator** — Manage build pipelines
- **Contributors** — Contribute to code and work items
- **Readers** — View code, work items, and reports
- **Project Valid Users** — Basic access to the project

Permissions Scopes

- **Organization** — Affects all projects
- **Project** — Affects only one project
- **Resource Level** — Fine-grained control (e.g., repo, pipeline)

Grant the least privilege necessary. Review permissions regularly.

5 Access Levels

Control how users access the organization.

Access Level	Description
Basic	View and contribute to public projects
Stakeholder	Advanced access to private features
Basic + Test Plans	Includes test case management
Visual Studio Subscriber	Full access to all features
Project Collection Administrators	Full management of the organization

Choose the right license to match your team's needs.

6 Project Settings

Configure how your project works.

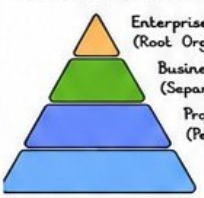
General Settings	Name, description, avatars
Visibility	Public or Private
Process	Agile, Scrum, CMMI, Basic
Repositories	Default Git settings
Pipelines	Default pipeline behaviors
Boards	Work item types and rules
Permissions	Project-level security

Pro Tip

- → Set meaningful project name and description.
- → Choose the right process template early.
- → Configure repository policies and protections.
- → Enable auditing and logging.

7 Enterprise Organization

For large enterprises, structure for scale and governance.



- ☑ Separate organizations for business units
- ☑ Align with governance and compliance needs
- ☑ Central identity and policy management
- ☑ Standardize processes, templates, and tooling

Plan your structure before creating projects.
Restructuring later is complex.

8 Best Practices

- ☑ Use meaningful names for organizations and projects.
- ☑ Follow naming conventions.
- ☑ Group users into teams and assign permissions to teams.
- ☑ Apply least privilege access.
- ☑ Review permissions and access regularly.
- ☑ Enable MFA for all users.
- ☑ Use service accounts for automation.
- ☑ Document your structure and access model.
- ☑ Monitor audit logs for changes and activity.

Remember

Good structure and permissions today prevent security risks and operational pain tomorrow.

Strong organizations. Well-structured projects. Secure access. Scalable delivery.
Plan it right. Build it clean. Operate it with confidence.

VERIQTA

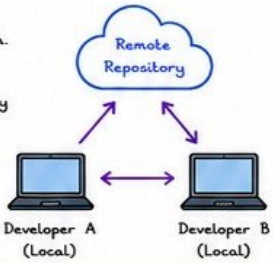
3. Azure Repos
Git Fundamentals

Azure Repos is the source control solution in Azure DevOps.
Built on Git. Fast. Secure. Scalable.

① Git Fundamentals

Git is a distributed version control system.
Key concepts:

- Repository – stores your project history
- Clone – get a local copy
- Commit – save changes
- Branch – work in isolation
- Merge – combine changes
- Remote – shared repository



Commit early. Commit often. Write meaningful messages.

② Repositories

A repository (repo) contains all files and history.
Types in Azure Repos:

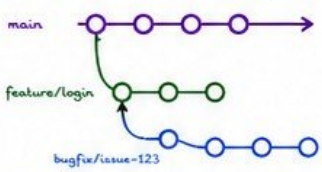
Git Repositories
- ☑ Distributed
- ☑ Branching & merging
- ☑ Offline work
- ☑ Best for most projects

Other Repository Types
- ☑ TFVC (for legacy Team Foundation Version Control)
- ☑ Git LFS (Large File Storage)

Use Git repositories for modern DevOps workflows.

③ Branches

Branches allow parallel development.



- ☑ main – stable production code
- ☑ feature/* – new features
- ☑ bugfix/* – bug fixes
- ☑ release/* – prepare release
- ☑ hotfix/* – urgent fixes

Short-lived branches. Merge often.

④ Commits

A commit records changes to the repository.

Best Practices
- ☑ Write clear messages
- ☑ One logical change per commit
- ☑ Reference work items (AB#123)
- ☑ Don't commit secrets

⑤ Pull Requests (PR)

PRs enable code review and collaboration.

PR = Discussion + Review + Quality Gate

⑥ Merge Strategies

Choose the right strategy for your team.

Merge Commit
Preserves history.
Default and safe.

Squash and Merge
Clean history.
Good for features.

Rebase and Merge
Linear history.
Use with caution.

For production, prefer Squash or Rebase for clean history.

⑦ Repository Policies

Policies enforce quality and security.

- ☑ Require PR reviews
- ☑ Minimum reviewers
- ☑ Build validation
- ☑ Status checks (CI/CD)
- ☑ Branch permissions
- ☑ Limit push to main
- ☑ Work item linking
- ☑ Comment requirements

Policies protect your code.
Enable them early.

⑧ Production Workflows

Example: Trunk-Based Workflow

- ☑ Small changes frequently
- ☑ Automated CI/CD for every commit
- ☑ Quality gates before production
- ☑ Rollback ready at every stage

Automate everything. Manual steps are the bottleneck.

⑨ Best Practices

- ☑ Use meaningful branch strategy
- ☑ Protect main branch
- ☑ Enforce PR reviews
- ☑ Automate builds and tests
- ☑ Keep history clean
- ☑ Secure secrets (no hardcoding)
- ☑ Monitor repository activity
- ☑ Document your workflows

Remember:
Good source control = Reliable releases + Happy teams

Good teams follow process. Great teams automate it.
Plan. Code. Commit. Review. Test. Deploy. Monitor.

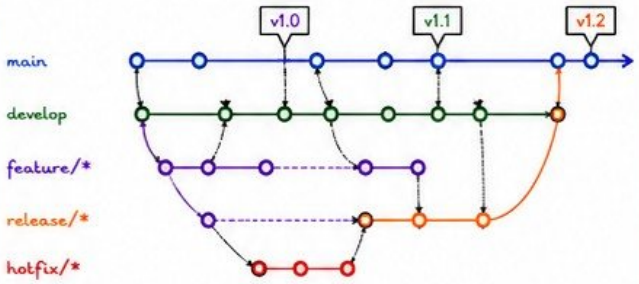
VERIQTA

4. Branching Strategies

A good branching strategy keeps your codebase stable, your team productive, and your releases predictable.

1 Git Flow

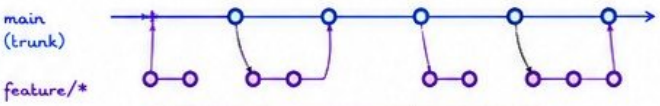
Best for: Libraries, products with scheduled releases



Stable releases from release/* merged to main.
Hotfixes come from main.

2 Trunk-Based Development

Best for: Continuous delivery environments

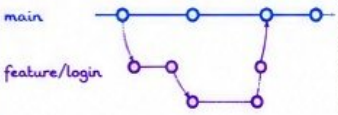


- [x] Short-lived feature branches
- [x] Frequent commits to main
- [x] Automated builds and tests
- [x] Small, safe changes
- [x] Fast feedback and delivery

Keep main always deployable.
Automate everything.

3 Feature Branches

Best for: New features, experiments



- [x] Created from main or develop
- [x] Isolated development
- [x] Merge back via PR
- [x] Delete after merge

Keep branches small and focused.
One feature = One branch.

4 Release Branches

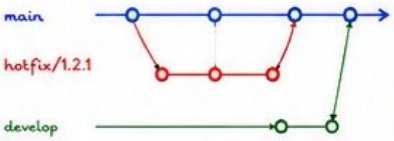
Best for: Preparing a release

- [x] Created from develop
- [x] Bug fixes and polish only
- [x] No new features
- [x] Merge to main and develop

Protect the release branch.
Only allow fixes.

5 Hotfix Branches

Best for: Urgent production fixes



- [x] Created from main
- [x] Fix critical issues
- [x] Merge back to main
- [x] Alse merge to develop
- [x] Tag a new version

Speed is critical. Keep hotfixes minimal.
Test and release fast.

6 Branch Protection

Protect important branches from mistakes.

- [x] Protect main, develop, release/*
- [x] Require PR reviews
- [x] Require status checks to pass
- [x] Require up-to-date branches
- [x] Disable force push
- [x] Limit who can merge

Example (Main Branch)

7 Merge Policies

Enforce quality with merge rules.

- [x] Require pull request
- [x] Minimum number of reviewers
- [x] Build validation
- [x] Automated tests
- [x] Security scanning
- [x] Branch policy enforcement
- [x] Work item linking

Azure DevOps Policy Types

8 Enterprise Recommendations

- [x] Choose a strategy that fits your delivery model.
- [x] Keep main stable and deployable.
- [x] Automate builds, tests, and quality gates.
- [x] Use branch policies to enforce standards.
- [x] Review and clean up branches regularly.
- [x] Document your workflow and train your teams.
- [x] Continuously improve your process.

Remember:

A clear strategy
+ strong policies
= reliable delivery
+ happy teams.

Good branching is not about Git commands. It's about teamwork, clarity, and quality.

Plan it. Protect it. Automate it. Deliver it.

VERIQTA

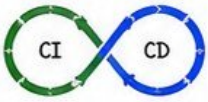
5. Azure Pipelines Fundamentals

Azure Pipelines is a CI/CD service that helps teams build, test, and deliver code faster and more reliably.

① CI/CD Concepts

CI (Continuous Integration)

Developers integrate code frequently.
Automated builds and tests validate changes.



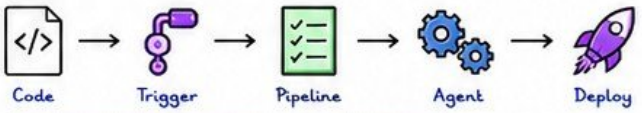
CD (Continuous Delivery / Deployment)

- Continuous Delivery: Code is always ready to deploy.
- Continuous Deployment: Code is automatically deployed to production.

CI gives you confidence in your code.
CD gets your code to your customers.

② Pipeline Architecture

A pipeline is defined in YAML and executed by agents.



- ☑ Code changes trigger the pipeline.
- ☑ Pipeline runs steps in order.
- ☑ Agent executes the jobs.
- ☑ Artifacts are produced and deployed.

Pipelines automate the path from code to production.

③ Pipeline Agents

Agents run your pipeline jobs.

Microsoft-Hosted Agents

Managed by Microsoft.
Always up-to-date.
Available on Windows, Linux, and macOS.

Self-Hosted Agents

You manage and maintain.
Custom configurations.
Run behind firewalls or in private networks.

Choose the right agent based on your security, compliance, and performance needs.

④ Hosted Agents

Microsoft-hosted agents are ready to use.

Supported Images

- ☑ No setup or maintenance.
- ☑ Automatically scaled.
- ☑ Clean environment for every job.
- ☑ Great for most scenarios.

Ideal for public projects and standard workloads.

⑤ Self-Hosted Agents

More control. More responsibility.

- ☑ Install on your own machines or VMs.
- ☑ Reuse tools, SDKs, and dependencies.
- ☑ Run in private networks.
- ☑ Scale with your infrastructure.
- ☑ Requires maintenance and updates.

Best for enterprise, legacy systems, and high-security environments.

⑥ Pipeline Stages

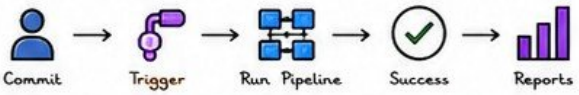
Divide your pipeline into logical stages.

- ☑ Stages run in sequence (or in parallel).
- ☑ Add approvals and checks between stages.
- ☑ Deploy with confidence.

Stages bring structure and control to your delivery process.

⑦ Pipeline Execution

How a pipeline runs:



- ☑ Real-time logs and status.
- ☑ Retry and re-run failed jobs.
- ☑ Artifacts and test results published.
- ☑ Notifications via email, Teams, or webhooks.

Visibility is key to fast feedback and reliable delivery.

⑧ Production Examples

Where Azure Pipelines shines in production.

- Microservices CI/CD
- Infrastructure as Code
- Web App Deployments
- Automated Testing
- Security Scanning
- Release Management
- Multi-Environment Deployments
- Blue/Green & Canary Deployments

Automate. Standardize. Secure. Scale.
Deliver value, not just code.

Good pipelines are reliable, repeatable, and observable.
Build it right. Test it well. Deploy it often. Monitor it always.

VERIQTA

6. YAML PIPELINES

YAML IS CODE. PIPELINES ARE AUTOMATION.

YAML pipelines define your CI/CD process as code.
Version it. Review it. Reuse it. Scale it.

① YAML SYNTAX

YAML is indentation-sensitive.

SYNTAX RULES

- [x] Use spaces, not tabs
- [x] Indent consistently (2 spaces recommended)
- [x] Key: Value pairs
- [x] Lists use '- '
- [x] Comments start with #

EXAMPLE

```yaml
# This is a comment
name: Build
steps:
  - script: echo Hello
    displayName: Say Hello
```

Wrong indentation = Pipeline failure.

② PIPELINE STRUCTURE

A basic pipeline has these top-level components.

```yaml
trigger:
  - main

variables:
  buildConfiguration: 'Release'

stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - script: echo "Building..."
```

- Trigger — When to run
- Variables — Values used in the pipeline
- Stages — Logical phases
- Jobs — Units of work
- Steps — Tasks or scripts

Everything in YAML is declarative. You define WHAT, not HOW.

③ TRIGGERS

Define when your pipeline runs.

CI TRIGGER (Continuous Integration)

Runs on code changes.

```yaml
trigger:
  branches:
    include:
      - main
      - develop
```

PR TRIGGER (Pull Request)

Runs when a PR is created/updated.

```yaml
pr:
  branches:
    include:
      - main
```

Use triggers wisely to avoid unnecessary builds.

④ VARIABLES

Store values and reuse them across the pipeline.

DEFINE VARIABLES

```yaml
variables:
  buildConfiguration: 'Release'
  vmImage: 'ubuntu-latest'
  majorVersion: '1'
```

USE VARIABLES

```yaml
- script: echo "Version $(majorVersion)"
- task: UseDotNet@2
  inputs:
    packageType: 'sdk'
    version: '8.x'
```

Use variables for flexibility and maintainability.

⑤ STAGES

Organize your pipeline into logical phases.


```yaml
stages:
  - stage: Build
  - stage: Test
  - stage: Staging
  - stage: Production
```

- [x] Stages run in sequence (by default)
- [x] Add approvals between stages
- [x] Enable environment promotions

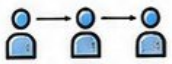
Stages provide structure, visibility and control.

⑥ JOBS

Jobs run in a stage and can run in parallel.

```yaml
stages:
  - stage: Build
    jobs:
      - job: BuildJob
        displayName: 'Build App'
        pool:
          vmImage: 'ubuntu-latest'
      - job: UnitTestJob
        displayName: 'Run Unit Tests'
        pool:
          vmImage: 'ubuntu-latest'
```

- [x] Jobs run in parallel by default
- [x] Use dependsOn to control order
- [x] Use conditions for control
- [x] Each job runs on an agent



Parallel Jobs

Keep jobs small, focused and independent.

⑦ STEPS

Steps are the tasks executed in a job.

```yaml
steps:
- checkout: self
- task: UseDotNet@2
  inputs:
    packageType: 'sdk'
    version: '8.x'
- script: dotnet build
  displayName: 'Build Solution'
- script: dotnet test
  displayName: 'Run Tests'
```

- checkout: Get your code
- task: Pre-built tasks
- script: Run scripts/commands
- template: Reuse YAML
- powershell/bash: Custom logic

Order matters. Steps run sequentially.

⑧ PRODUCTION BEST PRACTICES

- [x] Store pipelines in version control
- [x] Use meaningful names and comments
- [x] Use variables and variable groups
- [x] Secure secrets with Azure Key Vault
- [x] Use templates to keep YAML DRY
- [x] Enable PR validation builds
- [x] Use approvals for production stages
- [x] Pin task versions (avoid floating)
- [x] Monitor pipeline performance
- [x] Keep pipeline simple and readable

Good pipelines are secure, reliable and easy to maintain.

QUICK REFERENCE

Trigger on main:

```yaml
trigger:
  - main
```

Use a variable:

```
echo "$(variableName)"
```

Use a template:

```yaml
- template: build.yml
  parameters:
    configuration: 'Release'
```

Build once. Use everywhere.

YAML makes your pipeline repeatable, reviewable and reliable.
Automate the Right Way. Deliver with Confidence.

VERIQTA

7. Pipeline Variables and Parameters

Variables and parameters make your pipelines dynamic, reusable and secure.
Use them wisely. Protect them always.

① Variables

Variables store values used in your pipeline.

```yaml
# YAML Example
variables:
  - name: buildConfiguration
    value: 'Release'
  - name: vmImage
    value: 'ubuntu-latest'
  - name: nodeVersion
    value: '18.x'
```

- ☑ Simple key/value pairs
- ☑ Used in scripts, tasks, conditions
- ☑ Can be overridden at queue time
- ☑ Scope: Pipeline, Stage, Job

Keep variable names clear and consistent.
Use camelCase or PascalCase.

② Secret Variables

Store sensitive data securely.

```yaml
variables:
  - name: mySecret
    value: $(MY_SECRET)
    isSecret: true
```

- ☑ Encrypted at rest
- ☑ Masked in logs
- ☑ Set in the UI
- ☑ Never hardcode secrets
- ☑ Use secret variables for tokens, keys, passwords

Never echo secret variables in scripts.
Always use them through tasks or environment variables.

③ Variable Groups

Group variables for reuse across pipelines.

Create in Library

Pipelines > Library > Variable groups > New variable group

Example Group: CommonVariables

- ☑ appName = MyWebApp
- ☑ environment = production
- ☑ connectionString = ★ (secret)
- ☑ dockerRegistry = myregistry.azurecr.io

Share with multiple pipelines

Use variable groups for consistency and maintainability.
Limit access using permissions.

④ Runtime Parameters

Parameters are set at queue time. They are not variables.

```yaml
parameters:
  - name: deployEnvironment
    displayName: 'Select environment'
    type: string
    default: 'dev'
    values:
      - dev
      - stage
      - prod
```

- ☑ Strongly typed
- ☑ Visible at run time
- ☑ Cannot be changed during execution
- ☑ Best for control flow and decisions

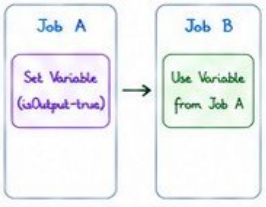
Use parameters for choices, flags, and deployment targets.

⑤ Output Variables

Pass data between jobs and stages.

```yaml
# Set output variable
echo "##vso[task.setvariable variable=buildIdFromBuild;isOutput=true]$(Build.BuildId)"

# Use in another job
variables:
  buildIdFromBuild: $[ dependencies.BuildJob.outputs['SetVars.buildId'] ]
```



Output variables help share information across jobs/stages.
Great for dynamic values.

⑥ Environment Variables

Expose variables as environment variables to tasks and scripts.

```yaml
steps:
  - script: |
      echo "App: $APP_NAME"
      echo "Env: $ENVIRONMENT"
    env:
      APP_NAME: $(appName)
      ENVIRONMENT: $(environment)
```

Available to scripts and tools

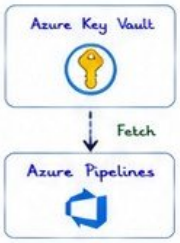
Keeps scripts portable and secure

Use environment variables to pass values to scripts/tasks.
Avoid hardcoding.

⑦ Secure Variable Management

Protect secrets and sensitive information.

- ☑ Use Azure Key Vault integration
- ☑ Limit access to variable groups
- ☑ Use secret variables for sensitive data
- ☑ Rotate secrets regularly
- ☑ Audit access to variables
- ☑ Never commit secrets to Git



Integrate Key Vault for maximum security and control.

⑧ Production Examples

Real world usage patterns.

Multi-Environment

Use parameter to select environment (dev, test, prod) and deploy with the same pipeline.

Secrets Management

Store DB passwords, tokens, and API keys as secret variables or in Key Vault.

Reusable Pipelines

Use variable groups and parameters to share pipelines across projects and teams.

Build once. Configure everywhere. Secure always.

Great pipelines are built with parameters, protected with secrets, and powered by the right variables. Be Smart. Be Secure. Be Reliable.

VERIQTA

8. Build Pipelines

Build. Test. Package. Deliver Confidence.

Build pipelines convert source code into reliable, tested, and versioned artifacts ready for deployment.
Automate the boring. Validate everything.

① Source Checkout

Get your code from the repository.

```yaml
steps:
- checkout: self
  clean: true
  fetchDepth: 0
  submodules: true
```

- ☑ Clean workspace
- ☑ Fetch full history (when needed)
- ☑ Include submodules
- ☑ Use shallow clone for performance

Clean checkouts prevent build issues from old artifacts and files.

② Restore Dependencies

Restore all required packages and libraries.

.NET Example

```yaml
- task: DotNetCoreCLI@2
  displayName: 'Restore packages'
  inputs:
    command: 'restore'
    projects: '**/*.sln'
```

Node.js Example

```yaml
- script: npm ci
  displayName: 'NPM Install'
```

- ☑ Use lock files (packages.lock.json, package-lock.json)
- ☑ Fail build if restore fails
- ☑ Cache dependencies for speed

Consistent dependencies = consistent builds.

③ Build Applications

Compile and build your application.

.NET Example

```yaml
- task: DotNetCoreCLI@2
  displayName: 'Build'
  inputs:
    command: 'build'
    projects: '**/*.sln'
    arguments: '--configuration $(BuildConfiguration)'
```

Maven Example

```yaml
- task: Maven@3
  displayName: 'Maven Build'
  inputs:
    mavenPomFile: 'pom.xml'
    goals: 'clean package'
```

- ☑ Use Release configuration for production builds
- ☑ Fail fast on compilation errors
- ☑ Enable parallel builds where possible

Fast, clean, and reproducible builds save time and reduce risk.

④ Run Tests

Automated tests catch issues early.

.NET Unit Tests

```yaml
- task: DotNetCoreCLI@2
  displayName: 'Test'
  inputs:
    command: 'test'
    projects: '**/*Tests/*.csproj'
    arguments: '--configuration $(BuildConfiguration)--
      --collect:"XPlat Code Coverage"'
```

- ☑ Run unit, integration, and component tests
- ☑ Collect code coverage
- ☑ Fail build on test failures
- ☑ Publish test results

No tests, no confidence. Automate test execution.

⑤ Publish Artifacts

Share build outputs with the rest of the pipeline.

```yaml
- task: PublishBuildArtifacts@1
  displayName: 'Publish Artifacts'
  inputs:
    PathtoPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'drop'
    publishLocation: 'Container'
```

Artifact Types

- Build outputs (binaries)
- Packages (NuGet, NPM)
- Docker images
- Logs & reports

Artifacts create traceability and enable deployment.

⑥ Build Validation

Validate the build before publishing.

- ☑ All tests passed
- ☑ Code coverage meets threshold
- ☑ No security vulnerabilities
- ☑ Static code analysis passed
- ☑ No warnings (or acceptable level)
- ☑ Quality gates passed

Quality Tools

- SonarQube
- ESLint
- OWASP Dependency Check
- CodeQL

Quality gates stop bad code from moving forward.

⑦ Build Optimization

Make your builds faster and more efficient.

- ☑ Enable pipeline caching
- ☑ Use parallel jobs
- ☑ Avoid unnecessary steps
- ☑ Use incremental builds
- ☑ Limit scope with path filters
- ☑ Use self-hosted agents for heavy builds

Cache Example (NPM)

```yaml
- task: Cache@2
  inputs:
    key: 'npm | "$(Agent.OS)" |
         package-lock.json'
    path: '$(Pipeline.Workspace)/.npm'
    restoreKeys: |
       npm | "$(Agent.OS)"
```

Optimize for speed without sacrificing reliability.

⑧ Production Pipelines Example

Typical enterprise build pipeline.

Checkout → Restore → Build → Test → Publish → Quality Gate

- ☑ Automated from commit to artifact
- ☑ Repeatable, reliable, and traceable
- ☑ Fast feedback for developers
- ☑ Ready for deployment or release stage

Build once.
Trust always.
Deliver value.

A great build pipeline is the foundation of great releases.
Automate. Validate. Optimize. Deliver. Repeat.

VERIQTA

9. Release Pipelines

Deliver Software Safely to Production.

Release pipelines take built artifacts and deploy them to environments in a controlled, repeatable, and auditable way.
Automate delivery. Protect quality. Reduce risk.

1 Deployment Stages

Organize your release into logical stages.

Dev → Test → Staging → Production

```
stages:
  - stage: Dev
  - stage: Test
  - stage: Staging
  - stage: Production
```

- [x] Stages run in order
- [x] Each stage has approvals & checks
- [x] Promote artifacts between stages
- [x] Isolate environments

Stages bring structure, control, and visibility to your deployments.

2 Environments

Environments represent real targets for deployments.

Dev · Test · Staging · Production

Environment details:
- [x] Resources
- [x] History
- [x] Approvals & checks
- [x] Linked pipelines

Example Environment Types
- Virtual machines
- Kubernetes
- App Service
- On-premises servers
- Multiple regions

Use environments to model real-world infrastructure.
Track and govern deployments.

3 Deployment Approvals

Add human checkpoints before deployments continue.

Pre-deployment approvals
- [x] Require approval
- [x] Per environment
- [x] Role-based
- [x] Time-bound

Approver Options
- Specific users
- Groups
- Dynamic (variable based)
- Business hours only

Approvals add human control where automation should pause and confirm.

4 Gates (Pre & Post Deployment Checks)

Automated quality checks before and after deployments.

Pre-deployment gates
- [x] Validate tests passed
- [x] Security scan passed
- [x] Service health check
- [x] Business hours check

Post-deployment gates
- [x] Smoke tests
- [x] API health check
- [x] Performance checks
- [x] Monitoring alerts

Gate Types
- Approval
- Query Azure Monitor
- REST API Check
- Invoke Azure Function
- Business Hours

Gates prevent bad deployments.
Tail fast before or after going live.

5 Rollback

Be ready to roll back quickly and safely.

Detect issue → Stop traffic → Rollback → Verify

```
Rollback options:
  - Redeploy last known good
  - Use deployment history
  - Reverse traffic (Blue/Green)
  - Scale down new version
  - Restore database (if needed)
```

Keep it simple.
Automate rollback steps where possible.

Fast rollback = minimal impact.
Always test your rollback process.

6 Release Strategies

Choose the right strategy for your scenario.

Recreate	Rolling	Blue / Green	Canary
Stop old Deploy new Simple but downtime	Update in batches Zero downtime safe updates	A → B Switch traffic from Blue to Green Instant rollback	Release to a % of users first Reduce risk

Match the strategy to your app and business risk.

7 Deployment History

Every deployment is tracked and auditable.

- View all deployments
- Status & outcome
- Artifacts used
- Changes & comments
- Who triggered it

Example History View

Release	Environment	Status	By	When
102	Production	✓ Succeeded	Ann	Today 10:15
101	Staging	✓ Succeeded	Ben	Today 09:02
100	Test	✗ Failed	Ann	Yesterday
99	Dev	✓ Succeeded	Ben	Yesterday

History gives visibility, accountability, and traceability.

8 Production Deployments

Best practices for deploying to production.

- [x] Automate everything you can
- [x] Use approvals & gates
- [x] Deploy small and validate
- [x] Monitor closely after release
- [x] Document every release
- [x] Test rollback regularly
- [x] Keep infrastructure as code
- [x] Communicate with stakeholders

Production Checklist
- [x] Tests passed
- [x] Security scan passed
- [x] Approvals completed
- [x] Gates passed
- [x] Monitoring ready
- [x] Rollback plan ready

Production is not the time to take risks.
Prepare, validate, and deliver with confidence.

Great releases are not accidental. They are automated, validated, and protected.
Plan it. Test it. Approve it. Deploy it. Monitor it. Improve it.

VERIQTA

10. Pipeline Security

Secure Your Pipelines. Protect Your Production.

Security is everyone's responsibility. In pipelines, one weak secret can compromise your entire environment. Secure by design.

① Service Connections

Connect Azure DevOps to external services securely.

Types

- Azure Resource Manager
- AWS
- GCP
- Kubernetes
- Other services

Best Practices

- [x] Use dedicated service connections
- [x] Grant least privilege access
- [x] Avoid using personal credentials
- [x] Rotate credentials regularly
- [x] Review access periodically

Service connections should have only the permissions required for pipeline execution.

② Secure Files

Store sensitive files (certificates, keys, config) securely.

Examples

- Certificates (.pfx, .crt)
- Private keys (.pem, .key)
- Config files (config.json)

Best Practices

- [x] Upload to Secure Files library
- [x] Reference in pipeline using secure download
- [x] Limit access to specific pipelines
- [x] Audit who has access

Never store sensitive files in repositories.

③ Secrets (Variables)

Keep secrets out of code and logs.

```
variables:
  mySecret: $(MySecret)

steps:
- script: echo "$(MySecret)"
  displayName: Use secret
```

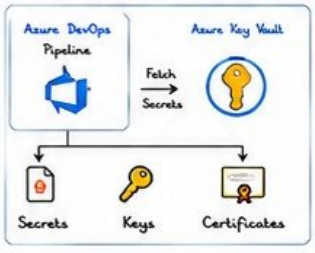
Best Practices

- [x] Mark variables as secret
- [x] Secrets are masked in logs
- [x] Do not print secrets
- [x] Use secret variables for tokens, keys, passwords

If you see a secret in logs, it's already compromised.

④ Azure Key Vault

Centralized secrets management for enterprises.



Best Practices

- [x] Use managed identity for access
- [x] Disable secret versions if not needed
- [x] Enable soft delete and purge protection
- [x] Rotate secrets periodically
- [x] Audit access logs

Key Vault + Managed Identity = Secure and Scalable.

⑤ Least Privilege

Grant minimum permissions required to succeed.

✗ Too Much Access	✓ Least Privilege
Contributor (Full Access)	Reader / Resource Specific Role
↓	↓
High Risk	Low Risk

Principles

- [x] Use custom roles when possible
- [x] Scope access to resource groups, not subscriptions
- [x] Regularly review and reduce permissions

More access = More risk. Less access = More safety.

⑥ Pipeline Permissions

Control who can run, edit, and manage pipelines.

Permissions

- Edit pipeline
- Queue build
- Manage releases
- View secrets
- Administer

Best Practices

- [x] Use groups, not individuals
- [x] Separate duties for security
- [x] Restrict edit access to trusted users
- [x] Review permissions regularly

Principle of least privilege applies to people too.

⑦ Credential Management

Handle credentials safely throughout the pipeline.

What to Avoid	What to Do
☒ Hardcoding credentials	☑ Use service connections
☒ Storing in source code	☑ Use Key Vault secrets
☒ Using personal accounts	☑ Use short-lived tokens
☒ Sharing credentials	☑ Rotate credentials
☒ Long-lived credentials	☑ Use managed identities

Credentials should be invisible in code and logs.

⑧ Secure Pipeline Execution

Harden your pipeline runtime environment.

Best Practices

- [x] Use latest hosted agent images
- [x] Enable pipeline workspace cleanup
- [x] Disable script debugging in production
- [x] Use self-hosted agents behind firewalls
- [x] Limit outbound network access
- [x] Scan dependencies for vulnerabilities

A secure build environment protects your code and infrastructure.

⑨ Audit and Monitoring

Visibility helps detect and respond to threats.

Monitor

- Pipeline runs
- Access changes
- Variable access
- Key Vault access
- Service connections

Tools

- Azure DevOps Audit Logs
- Azure Monitor
- Microsoft Sentinel
- Key Vault Logs

Monitor. Alert. Respond. Repeat.

⑩ Enterprise Security Checklist

- [x] Service connections use least privilege
- [x] Secrets stored in Key Vault
- [x] No secrets in code or logs
- [x] Secure files used for certificates/keys
- [x] Pipeline permissions reviewed
- [x] Approvals & gates for production
- [x] Audit logs enabled
- [x] Service connections reviewed quarterly
- [x] Credentials rotated regularly
- [x] Least privilege enforced everywhere
- [x] Self-hosted agents hardened
- [x] Dependencies scanned
- [x] Secure pipeline templates
- [x] Branch protections enforced
- [x] Incident response plan in place

Security is not a feature. It's a foundation.

Secure pipelines build trust. Trust enables speed. Speed delivers value.
Secure Today. Automate Safely. Deliver with Confidence.

VERIQTA

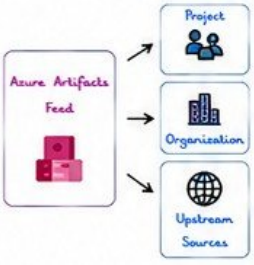
11. Azure Artifacts

Manage, Share, and Secure Packages.

Azure Artifacts is your private package management solution.
One place for all your packages. Fast, reliable, and secure.
Version it. Control it. Deliver it.

① Package Feeds

A package feed stores and manages your packages.



- Private and secure
- Role-based access control
- Upstream sources
- Retention policies
- Audit and logs
- Integration with pipelines

One feed = one source of truth for your packages.

② NuGet Packages

Manage .NET packages with NuGet.

```
# Add feed to NuGet.config
[packageSources]
  <add key="veriqta-feed"
       value="https://pkgs.dev.azure.com/
your-org/_packaging/your-feed/nuget/v3/index.json"
  protocolVersion="3" />
# Install package
dotnet add package MyPackage
  --source veriqta-feed
```

- Push and pull packages
- SemVer versioning
- Dependency resolution
- Upstream (NuGet.org)
- Usage in pipelines

Use consistent versioning and pin exact versions.

③ NPM Packages

Manage JavaScript/TypeScript packages.

```
# .npmrc
registry=https://pkgs.dev.azure.com/
your-org/_packaging/your-feed/npm/
registry/
always-auth=true

# Install package
npm install my-package --registry
https://pkgs.dev.azure.com/
your-org/_packaging/your-feed/npm/
registry/
```

- Private npm registry
- Scoped packages
- Access tokens for auth
- Proxy to npmjs.org
- Works with CI/CD

Never commit tokens. Use .npmrc + secrets.

④ Maven Packages

Manage Java libraries and artifacts.

```
<!-- settings.xml -->
<servers>
  <server>
    <id>veriqta-feed</id>
    <username>$(username)</username>
    <password>$(PAT)</password>
  </server>
</servers>
<!-- pom.xml -->
<repository>
  <id>veriqta-feed</id>
  <url>https://pkgs.dev.azure.com/
  your-org/_packaging/your-feed/
  maven/v1/</url>
</repository>
```

- Hosted Maven repository
- Snapshot & release repos
- Dependency caching
- Upstream (Maven Central)
- Checksums & signatures

Separate snapshot and release repositories.

⑤ Python Packages

Manage Python packages (PyPI).

```
# Create .pypirc
[distutils]
 index-servers = veriqta

[veriqta]
repository = https://pkgs.dev.azure.com/
your-org/_packaging/your-feed/pypi/upload
username = __token__
password = $(PAT)

# Upload package
python -m twine upload dist/*
  -r veriqta
```

- PyPI compatible
- Upload with twine
- Access tokens for auth
- Virtual environments
- Version management

Use twine + tokens. Never store credentials in code.

⑥ Universal Packages

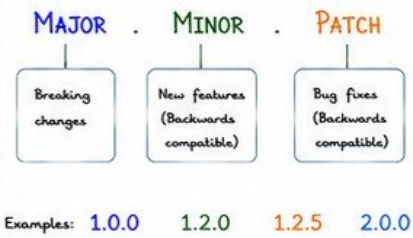
Store any type of artifact.

- Flexible storage
- Versioned packages
- Download via REST API
- Use in pipelines
- Cross-platform support

Great for tools, CLIs, and large assets.

⑦ Versioning

Follow semantic versioning (SemVer).



Examples: 1.0.0 1.2.0 1.2.5 2.0.0

Pin exact versions in production builds.

⑧ Production Package Management

Best practices for enterprise environments.

- Use private feeds for internal packages
- Enforce versioning and immutability
- Promote packages across environments
- Scan for vulnerabilities
- Restrict who can publish
- Enable retention policies
- Audit package usage
- Use upstream sources wisely
- Automate cleanup of old versions

Promote, Don't Rebuild. Ensure traceability.

⑨ Feed Security

Secure your feeds and packages.

- Use least privilege access
- Use service connections in pipelines
- Store tokens in secrets
- Enable audit logging
- Scan packages for vulnerabilities
- Disable anonymous access
- Review permissions regularly

Security is not optional. Build it in.

Good packages are versioned, tested, and trusted.

Version It. Share It. Secure It. Deliver It.

VERIQTA

12. Azure Boards

Plan Work. Track Progress. Deliver Value.

Azure DevOps Production Handbook
Volume 1

Azure Boards helps teams plan, track, and discuss work across the entire lifecycle.
From ideas to delivery — in one unified platform.
Visibility. Collaboration. Accountability.

① Work Items

Everything in Azure Boards is a work item.

Common Work Item Types

- [x] Epic
- [x] Feature
- [x] User Story
- [x] Task
- [x] Bug
- [x] Test Case
- [x] Issue

Example Work Item (User Story)

Title: As a user, I want to reset my password so that I can regain access to my account.

Acceptance Criteria:
- User can request reset email
- Email contains secure link
- Link expires in 30 minutes
- User can set a new password

State: Active
Assigned To: @dev

Work items bring structure and clarity to your work.

② User Stories

Capture user needs in simple, clear stories.

User Story Format (INVEST)

As a [type of user]
I want [an action]
So that [a benefit]

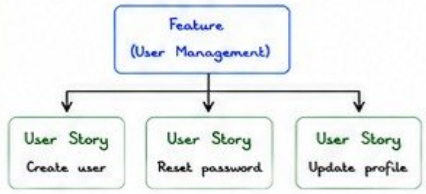
INVEST Principles

- [x] Independent
- [x] Negotiable
- [x] Valuable
- [x] Estimable
- [x] Small
- [x] Testable

Good stories drive good solutions.

③ Features

A feature is a collection of user stories.

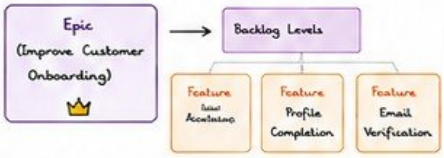


- [x] Features group related stories
- [x] Help plan and estimate
- [x] Delivered within a sprint or release

Features break big ideas into deliverable chunks.

④ Epics

Epics represent large bodies of work.

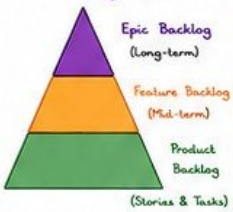


- [x] Epics are strategic and long-term
- [x] Help with budgeting and planning
- [x] Broken down into features

Epics connect strategy to execution.

⑤ Backlogs

Backlogs are prioritized lists of work.



- [x] Epic backlog = all big initiatives
- [x] Feature backlog = planned features
- [x] Product backlog = stories ready to build

Prioritize ruthlessly. Focus on value.

⑥ Sprint Planning

Plan work in short, time-boxed iterations.

Sprint Planning Steps

1. Review sprint goal
2. Select top priority items
3. Break stories into tasks
4. Estimate effort
5. Commit to sprint

Best Practices

- [x] Time-box the meeting
- [x] Involve the whole team
- [x] Focus on value
- [x] Avoid over-commitment

A good plan today prevents chaos tomorrow.

⑦ Kanban Boards

Visualize work. Limit WIP. Deliver continuously.

- [x] Limit Work In Progress (WIP)
- [x] Make policies explicit
- [x] Visualize flow
- [x] Continuously improve

Flow > Chaos. WIP limits create focus.

⑧ Enterprise Project Management

Scale visibility and governance across teams.

- **Portfolios** — Track strategic initiatives
- **Programs** — Group related projects
- **Projects** — Deliver features and releases
- **Teams** — Execute and collaborate

- [x] Align work to business goals
- [x] Track progress with dashboards
- [x] Standardize processes
- [x] Control access and security
- [x] Automate reporting
- [x] Drive continuous improvement

Enterprise success = alignment + visibility + execution.

⑨ Boards Reports & Analytics

Measure progress. Improve outcomes.

- **Burndown Chart** — Track sprint progress
- **Cumulative Flow** — Visualize flow over time
- **Velocity** — Measure team throughput
- **Work Item Aging** — Identify stale items

- [x] Real-time dashboards
- [x] Custom reports
- [x] Power BI integration
- [x] Share insights
- [x] Make data-driven decisions

What gets measured gets improved.

⑩ Best Practices for Azure Boards

- Define clear goals and success metrics
- Use consistent work item types and templates
- Keep backlogs prioritized and groomed
- Plan realistic sprints and commitments
- Use WIP limits and track flow
- Communicate early and often
- Review metrics and improve continuously
- Secure projects and control access
- Automate processes and notifications
- Deliver value in every iteration

Great teams plan smart. Work together. Deliver value. Continuously improve.
Plan It. Track It. Build It. Ship It. Improve It.

VERIQTA

13. Azure Test Plans

Test Early. Test Often. Ship With Confidence.

Azure Test Plans helps teams test the right things, track quality, and deliver reliable software.

Plan tests. Execute tests. Track defects. Improve quality.

1 Test Cases

Test cases describe what to test and how.

Test Case Fields

- [x] Title
- [x] Description
- [x] Preconditions
- [x] Test Steps
- [x] Test Data
- [x] Expected Result
- [x] Priority
- [x] Tags

Example Test Case

Title: User can reset password
Preconditions: User exists
Steps:
1. Go to login page
2. Click "Forgot password"
3. Enter email
4. Click "Send reset link"

Expected: Email with reset link is sent successfully

Good test cases are clear, repeatable, and have expected results.

2 Test Suites

Group related test cases for better organization.

Suite Types

- Requirement-based
- Feature-based
- Component-based
- Risk-based
- Smoke Suite
- Regression Suite

Example Suite

Login Feature Suite

- Login - Valid User
- Login - Invalid User
- Login - Lockout
- Login - Remember Me
- Login - Password Reset

Suites keep tests organized and easy to manage.

3 Manual Testing

Human-driven testing to validate behavior.

Manual Testing Types

- Functional Testing
- UI / UX Testing
- Usability Testing
- Compatibility Testing
- Ad-hoc Testing

Best Practices

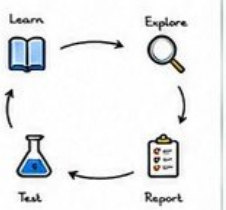
- [x] Follow test cases
- [x] Log results clearly
- [x] Attach evidence
- [x] Re-test after fix
- [x] Communicate issues

Manual testing finds what automation often misses.

4 Exploratory Testing

Learn, explore, and test without scripts.

How Exploratory Testing Works



When to Use

- [x] New features
- [x] Unclear requirements
- [x] Complex workflows
- [x] Before automation
- [x] Critical areas

Explore freely. Think like a user. Find the unexpected.

5 Test Execution

Run tests and record results.

Execution Flow

Select Test Suite → Run Test Cases → Record Results → Review Outcomes

Test Outcomes

- Passed
- Failed
- Blocked
- Not Executed

Execute consistently. Record accurately. Review objectively.

6 Bug Tracking

Report, track, and resolve defects.

Bug Lifecycle

New → Active → Resolved → Closed

Important Bug Fields

- [x] Title & Description
- [x] Steps to Reproduce
- [x] Expected Result
- [x] Actual Result
- [x] Severity
- [x] Priority
- [x] Environment
- [x] Attachments

A good bug report is complete, clear, and reproducible.

7 Test Reporting

Measure quality. Share insights.

Key Metrics

- Test Cases Designed
- Test Cases Executed
- Pass Rate (%)
- Defects Found
- Defects Closed

Report Types

- Test Plan Progress
- Suite Progress
- Test Case Results
- Defect Trend
- Requirement Coverage

Metrics make quality visible. Visibility drives action.

8 Quality Assurance Workflow

End-to-end QA process with Azure DevOps.

Plan → Design → Test → Execute → Track → Report → Improve

Best Practices

- [x] Involve QA early in the process
- [x] Review requirements and risks
- [x] Create traceable test cases
- [x] Automate repetitive tests
- [x] Track defects and trends
- [x] Continuously improve quality

Quality is a journey, not an event. Build quality in. Don't test it in.

9 Integration & Traceability

Link everything for full visibility.

Linking

Requirements → Test Cases → Test Suites → Bugs → Commits / Builds

Benefits

- [x] End-to-end traceability
- [x] Impact analysis
- [x] Better coverage
- [x] Faster releases
- [x] Audit ready

Traceability builds trust and ensures nothing is missed.

Great testing prevents small bugs from becoming big problems.

Plan It. Test It. Track It. Improve It.

VERIQTA

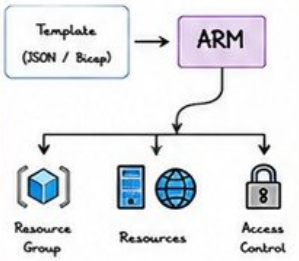
14. Deploying to Azure

Build It. Test It. Deploy It. Run It.

Azure gives you the flexibility, scale, and reliability to deploy your applications anywhere in the world.
Choose the right service. Configure correctly. Deploy with confidence.

① Azure Resource Manager (ARM)

ARM is the deployment and management service for Azure.



Benefits
- Infrastructure as Code
- Consistent deployments
- Repeatable
- Manage at scale
- Role-based access

Use resource groups to logically organize and manage all related resources.

② Web Apps (App Service)

Fully managed platform for web applications and APIs.

Azure App Service

Features
- Auto scale
- Built-in CI/CD
- Custom domains
- SSL/TLS
- Monitoring
- Easy deployments

Deploy from: GitHub · Azure Repos · Container Registry · ZIP Package

Use App Service for quick deployments with minimal operational overhead.

③ Virtual Machines (VM)

IaaS for full control over server environments.

Use Cases
- Legacy applications
- Custom configurations
- Lift and shift
- Development / testing
- Batch processing

Key Considerations
- OS updates & patching
- Backups
- Scaling
- Security hardening
- Monitoring

Use VM Scale Sets for high availability and auto scaling.

④ Azure Kubernetes Service (AKS)

Managed Kubernetes for containerized applications.

Capabilities
- Auto scaling
- Self-healing
- Managed control plane
- Azure AD integration
- Monitoring & logging
- Azure CNI networking

Best For
- Microservices
- Containerized apps
- Complex workloads

Deploy using: Helm · kubectl · GitOps (Flux/ArgoCD)

Use node pools to isolate workloads and optimize cost.

⑤ App Services (Beyond Web Apps)

Fully managed services for modern applications.

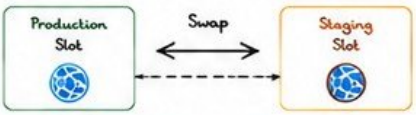
- Functions (Serverless)
- API Management (APIs)
- Logic Apps (Workflow)
- Service Bus (Messaging)

Benefits
- No server management
- Auto scaling
- High availability
- Pay for what you use
- Deep Azure integration

Use the right App Service based on your application pattern.

⑥ Deployment Slots (App Service)

Deploy with zero downtime using slots.



Benefits
- Test in production-like environment
- Instant rollback
- Safe deployments

Best Practices
- Keep configuration in sync
- Automate swaps
- Use warm-up settings

Always test in staging slot before swapping to production.

⑦ Production Deployments

Follow a safe and reliable deployment workflow.

Build → Test → Package → Deploy → Validate → Monitor

Strategies
- Rolling Update
- Blue / Green
- Canary Deployment
- Recreate
- Feature Flags

Guardrails
- Health checks
- Automated rollback
- Monitoring & alerts

Automate everything. Manual steps cause errors.

⑧ Best Practices

Security First
Use least privilege, secure secrets, and enable network security.

Cost Optimization
Right-size resources, use autoscale, and monitor costs.

High Availability
Design for redundancy and disaster recovery.

Observability
Monitor, log, and alert everything that matters.

Infrastructure as Code
Use Bicep/Terraform and version your infrastructure.

⑨ Deployment Checklist

- [x] Code committed and reviewed
- [x] Build and tests passed
- [x] Artifacts published
- [x] Infrastructure validated
- [x] Security scan completed
- [x] Deployment approved
- [x] Deployed to staging
- [x] Validated successfully
- [x] Deployed to production
- [x] Monitoring and alerts active
- [x] Rollback plan ready

Remember
A successful deployment is not just about releasing code. It's about delivering value reliably.

Great deployments are boring. Boring is good.

Plan It. Test It. Deploy It. Monitor It. Improve It.

VERIQTA

15. Production Deployment Strategies

Right Strategy. Right Risk. Right Outcome.

Azure DevOps Production Handbook
Volume 1
Page 15 of 20

Deployment is not just moving code —
it's delivering value safely.
Use proven strategies. Validate continuously.
Be ready to roll back.
Ship Fast. Stay Safe. Recover Faster.

1 Rolling Deployment

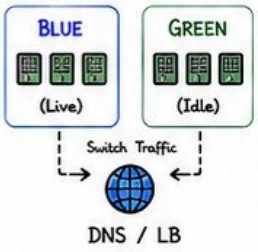
Gradually replace old version with new version.

Best For
- ☑ Stateless apps
- ☑ Kubernetes, VMs, App Services
- ☑ Simple and fast
- ☑ Low risk updates

Always monitor metrics during rollout.
Stop rollout on any health degradation.

2 Blue-Green Deployment

Maintain two identical environments.

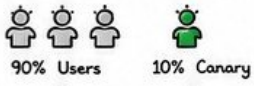


Best For
- ☑ High availability
- ☑ Large applications
- ☑ Fast rollback
- ☑ Minimal downtime

Test Green environment fully before switching traffic.

3 Canary Deployment

Release to a small subset of users first.



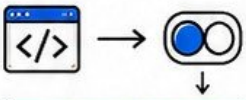
Monitor Metrics
- Errors
- Latency
- Performance
- Logs

Best For
- ☑ Risk reduction
- ☑ User-facing apps
- ☑ New features
- ☑ AI/ML services
- ☑ Gradual validation

Use metrics + user feedback to decide full rollout.

4 Feature Flags

Decouple deployment from release.



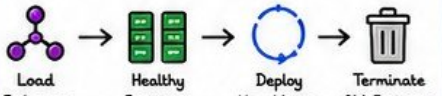
Control Feature
- Enable for certain users
- Enable by % rollout
- Enable by environment
- Instantly disable if needed

Benefits
- ☑ Release safely
- ☑ A/B testing
- ☑ Instant rollback
- ☑ No redeploy needed
- ☑ Grontrdr

Never remove flags immediately.
Plan for cleanup.

5 Zero Downtime Deployment

Ensure continuous availability during deployment.



Key Practices
- ☑ Health probes and readiness checks
- ☑ Graceful shutdown and connection draining
- ☑ Auto scaling to handle traffic
- ☑ Database migrations (backward compatible)
- ☑ Continuous monitoring

Design for resilience.
Downtime is not acceptable in production.

6 Rollback Strategy

Always have a safe exit plan.

Rollback Triggers

Automated Rollback
- Based on alerts
- Based on metrics
- Based on tests

Manual Rollback
- Redeploy previous
- Swap traffic (Blue-Green)
- Revert configuration

Test your rollback process.
A broken rollback is a bigger risk.

7 Health Validation

Validate before, during, and after deployment.

Validation Checklist
- ☑ Health probes (Liveness/Readiness)
- ☑ Smoke tests
- ☑ Integration tests
- ☑ Synthetic monitoring
- ☑ Database connectivity
- ☑ Dependency health
- ☑ Business KPIs

Tools
- Azure Monitor
- Application Insights
- Azure Load Testing
- Prometheus
- Grafana

Validate automatically.
Humans miss things.

8 Enterprise Deployment Workflow

A repeatable and secure end-to-end process.

Governance Layers

Standardize the process.
Automate the steps. Enforce governance.

9 Strategy Comparison

Strategy	Downtime	Risk	Complexity	Rollback Speed	Best For
Rolling	Low	Low	Low	Medium	Most Apps
Blue-Green	None	Low	Medium	Very Fast	Critical Apps
Canary	None	Very Low	Medium	Fast	User-Facing
Feature Flags	None	Very Low	Low	Instant	Features
Zero Downtime	None	Low	High	Depends	Enterpriss

No one strategy fits all.
Choose based on risk, value, and team maturity.

Production Deployment Best Practices

- ☑ Automate everything
- ☑ Use Infrastructure as Code (IaC)
- ☑ Keep builds and artifacts immutable
- ☑ Use versioned releases
- ☑ Deploy small and frequently
- ☑ Monitor every deployment
- ☑ Validate before promoting
- ☑ Use configuration as code
- ☑ Protect production environments
- ☑ Enforce approvals and gates
- ☑ Maintain audit trails
- ☑ Practice disaster recovery
- ☑ Test rollbacks regularly
- ☑ Document runbooks
- ☑ Continuously improve

Deployment is a skill. Reliability is a habit. Excellence is a choice.

Great teams deploy with confidence. Great engineers think before they ship.
Plan Smart. Deploy Safe. Monitor Always. Improve Continuously.

VERIQTA

16. Monitoring and Troubleshooting

Observe. Detect. Diagnose. Resolve.

You can't fix what you can't see.
Monitoring shows you what's happening.
Troubleshooting gets you to the root cause.
See it. Understand it. Solve it. Improve it.

1 Pipeline Logs

Understand what happened in your pipeline run.

Where to View

- Pipelines → Runs
- Select a run
- View logs for each job
- Download full logs

Logs are your first place to look. Always read top to bottom.

2 Deployment Logs

Check what happened during deployment.

Sources

- Azure App Service → Log Stream
- Azure Kubernetes Service → kubectl logs
- VM / Custom Script → Application logs
- Storage / Blob Logs
- Application Insights

Application logs tell you why your app failed.

3 Agent Diagnostics

Ensure your agent is healthy and not the problem.

Check

- ☑ Agent status (Online)
- ☑ Agent version
- ☑ Capabilities
- ☑ Demands / Permissions
- ☑ Disk space
- ☑ Network connectivity
- ☑ Agent logs

An unhealthy agent causes random, hard-to-debug failurers.

4 Build Failures

Build failed? Find the real reason.

Common Causes

- ✗ Compilation errors
- ✗ Missing dependencies
- ✗ Test failures
- ✗ Out of memory
- ✗ Wrong tool versions
- ✗ Script errors
- ✗ Permission issues

Read the error message. Don't guess. Don't skip steps.

5 Release Failures

Deployment failed? Identify the breaking point.

Common Causes

- ✗ Health check failures
- ✗ Configuration errors
- ✗ Missing secrets / permissions
- ✗ Service unavailable
- ✗ Database connection failure
- ✗ Wrong environment variables
- ✗ Approval or gate rejections

Check the deployment task that failed first.

6 Monitoring Deployments

Monitor your deployments in real time.

What to Monitor

- Deployment status
- Application health
- Performance metrics
- Error rate
- User experience
- Alerts and notifications

Good monitoring detects issues before users report them.

7 Root Cause Analysis

Find the real reason behind the problem.

RCA Process

1. Define the problem
2. Collect facts and data
3. Identify contributing factors
4. Find the root cause
5. Validate the cause
6. Implement permanent fix
7. Prevent recurrence

Fixing symptoms = temporary.
Fixing root cause = permanent.

8 Production Troubleshooting

A systematic approach to fix production issues fast.

Troubleshooting Workflow

Detect the issue
↓
Assess the impact
↓
Gather information
↓
Form a hypothesis
↓
Test and validate
↓
Mitigate / Resolve
↓
Review and improve

Stay calm. Follow the process. Communicate clearly.

Useful Commands & Tools

```
az pipelines runs list --top 10
az pipelines runs show --id <id>
az webapp log tail --name <app> --resource-group <rg>
az monitor metrics list --resource <id>
kubectl get pods,svc,deployments
kubectl logs <pod-name>
kubectl describe <resource>
```

Monitoring Tools

- Azure Monitor
- Application Insights
- Log Analytics
- Alerts
- Dashboards

Best Practices

- ☑ Enable detailed logging
- ☑ Monitor everything important
- ☑ Set meaningful alerts
- ☑ Test failure scenarios
- ☑ Document common issues
- ☑ Automate diagnostics
- ☑ Learn from every incident

Observability gives you visibility. Troubleshooting gives you reliability.

Great engineers prevent issues. Great teams detect issues early. Great leaders solve them permanently.

Observe. Detect. Diagnose. Resolve. Improve.

VERIQTA

17. Infrastructure as Code

Code It. Version It. Deploy It. Trust It.

Infrastructure as Code (IaC) brings consistency, speed, and reliability to cloud infrastructure.
Define. Automate. Deploy. Validate. Repeat.
No manual clicks. No drift. Full control.

1 ARM Templates

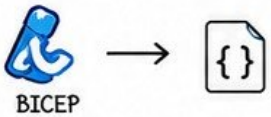
Azure Resource Manager (ARM) templates define your infrastructure as JSON.

Key Features
- Native Azure solution
- Declarative JSON syntax
- Supports all Azure resources
- Parameterization
- Modular with linked templates

Great for enterprise Azure environments. Highly integrated.

2 Bicep

Bicep is a simplified language for deploying Azure resources.

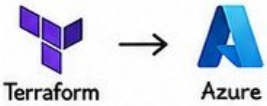


Key Features
- Human-friendly syntax
- Compiles to ARM template
- Modules and reusability
- Built-in functions
- Strong typing and validation

Write less code. More clarity. Fewer errors. Designed for Azure.

3 Terraform Integration

Use Terraform with Azure in Azure DevOps pipelines.



Benefits
- Multi-cloud support
- Strong community
- State management
- Reusable modules
- Extensible providers

Ideal for multi-cloud or hybrid environments.

4 Deployment Automation

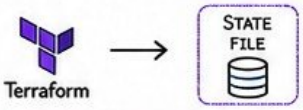
Automate infrastructure deployments using pipelines.

Automation Steps
1. Code commit
2. Validate
3. Plan / What-If
4. Approvals
5. Deploy
6. Verify

Automate everything. Reduce manual work. Increase speed.

5 State Management

State tracks the current infrastructure and maps real resources.



Best Practices
- Store state remotely
- Use Azure Storage / Backend
- Enable state locking
- Encrypt state files
- Restrict access

Good state management prevents conflicts and data loss.

6 Infrastructure Validation

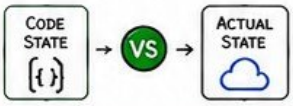
Validate templates before deployment to catch issues early.

Validation Techniques
- ARM Template What-If
- Bicep build & validate
- Terraform plan
- Linting and static analysis
- Policy as Code (OPA, PSRule)

Validate early. Save time. Prevent failures.

7 Drift Detection

Detect changes made outside of code to maintain consistency.



Detection Methods
- Azure Policy
- Azure Resource Graph
- What-If / Plan
- Continuous compliance scans
- Alert on deviations

No drift. No surprises. Stay compliant.

8 Enterprise Automation

Build scalable, secure, and repeatable infrastructure automation.

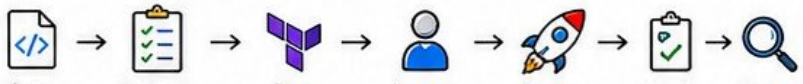
Enterprise Best Practices
- Standardize modules
- Use naming conventions
- Implement policies
- Use reusable pipelines
- Audit & monitor deployments
- Document everything

Standardization + Automation = Scale, Security, and Reliability.

IAc Toolkit

- **ARM Templates** — Native Azure IaC
- **Bicep** — Simplify ARM authoring
- **Terraform** — Multi-cloud IaC
- **Azure DevOps** — Automation & Pipelines
- **Azure Policy** — Governance & Compliance

Enterprise IaC Pipeline Example



Key Principles
- Infrastructure is Code
- Everything is versioned
- Automate the pipeline
- Validate before deploy
- Detect and fix drift
- Secure by design
- Reusable and modular
- Document and audit
- Continuously improve

Remember

IaC turns infra from manual and error-prone to consistent, repeatable, and scalable.

Great engineers automate infrastructure. Great architects standardize it. Great teams trust it.

Code It. Test It. Deploy It. Monitor It. Improve It.

VERIQTA

18. Azure DevOps Security

Secure It. Protect It. Monitor It. Trust It.

Security in Azure DevOps is a shared responsibility.
Everyone. Every action. Every pipeline. Every artifact.
Secure by design. Secure by default. Secure always.

1 RBAC (Role-Based Access Control)

Control who can do what in your organization.

Best Practices

- ☑ Use groups, not individuals
- ☑ Apply least privilege
- ☑ Review roles regularly
- ☑ Remove unnecessary access
- ☑ Separate duties

Grant only what is needed.
Nothing more. Nothing less.

2 Branch Security

Protect your codebase and enforce quality.

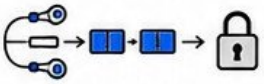
Controls

- ☑ Branch policies
- ☑ Require PR reviews
- ☑ Build validation
- ☑ Status checks
- ☑ Limit who can push
- ☑ Prevent force push

Protect main. Control changes.
Enforce reviews.

3 Pipeline Security

Secure your pipelines and execution environment.



Key Controls

- ☑ Limit pipeline permissions
- ☑ Use service connections with least privilege
- ☑ Protect YAML pipelines
- ☑ Approve sensitive tasks
- ☑ Use secure agents

Pipelines can deploy anything.
Lock them down.

4 Artifact Security

Protect your packages, feeds, and artifacts.

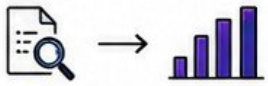
Best Practices

- ☑ Restrict feed permissions
- ☑ Use scoped access tokens
- ☑ Scan for vulnerabilities
- ☑ Sign and verify packages
- ☑ Retain and cleanup policies

Trust your artifacts.
Verify. Scan. Protect.

5 Audit Logs

Track activity and investigate changes.



Track & Monitor

- ☑ Audit streams
- ☑ User and admin actions
- ☑ Pipeline runs
- ☑ Security changes
- ☑ Service connections

Logs tell the truth.
Enable. Monitor. Act.

6 Compliance

Meet industry standards and regulatory needs.

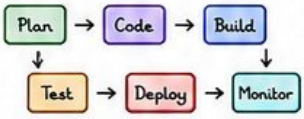
Focus Areas

- ☑ Access control
- ☑ Data protection
- ☑ Audit and logging
- ☑ Retention policies
- ☑ Third-party compliance

Document everything.
Compliance builds trust.

7 Secure Development Lifecycle (SDLC)

Build security into every stage of development.



Security in Every Stage

- ☑ Threat modeling
- ☑ Code scanning (SAST)
- ☑ Dependency scanning
- ☑ Secrets scanning
- ☑ DAST & Pen testing

Security is not a step.
It's a mindset.

8 Enterprise Recommendations

Build a strong, secure Azure DevOps platform.

Recommendations

- ☑ Use separate projects per team or product
- ☑ Enforce security baselines
- ☑ Use approvals for changes
- ☑ Monitor and alert
- ☑ Regular access reviews
- ☑ Train your teams

Strong security today.
Safe tomorrow.

Shared Security Controls

Azure DevOps Security Checklist

- ☑ RBAC reviewed and applied
- ☑ Branch policies enforced
- ☑ Pipelines secured and approved
- ☑ Service connections audited
- ☑ Artifacts scanned and protected
- ☑ Audit logs enabled and monitored
- ☑ Compliance requirements met

Security is not a feature.
It's a culture.
Everyone.
Every day.

Secure your platform. Protect your code. Safeguard your pipelines. Earn trust every day.

Secure It. Protect It. Monitor It. Trust It.

VERIQTA

19. Production Case Studies

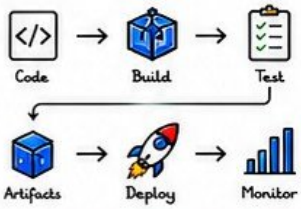
Learn From Real Scenarios. Apply Senior Thinking.

Azure DevOps Production Handbook
Volume 1

Real production is messy. Tools don't save you — process, visibility, and engineering judgment do.
These case studies show how senior engineers solve real problems in real systems.
Understand the why. Learn the how. Apply the lessons.

1 CI/CD for Microservices

Building and deploying microservices with Azure DevOps.



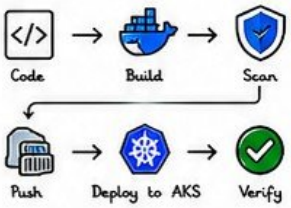
Key Practices

- ☑ Service isolation
- ☑ Independent pipelines
- ☑ Contract testing
- ☑ Versioned APIs
- ☑ Observability per service

Small services. Small failures. Fast feedback. Fast recovery.

2 AKS Deployment Pipeline

CI/CD pipeline for containerized apps on AKS.



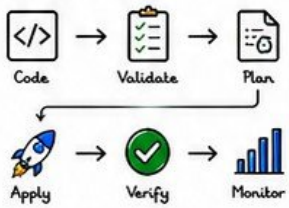
Key Practices

- ☑ Image scanning
- ☑ Helm charts
- ☑ Environment promotion
- ☑ Health probes & gates
- ☑ Auto rollback on failure

Secure images. Promote safely. Automate. Validate. Monitor.

3 Infrastructure Deployment

Infrastructure as Code pipeline using ARM/Bicep/Terraform.



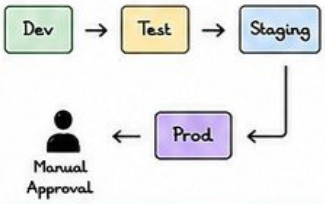
Key Practices

- ☑ Infrastructure in version control
- ☑ Plan and review changes
- ☑ Least privilege service connection
- ☑ Environment segregation
- ☑ Drift detection and alerts

No clicks. No drift. Full repeatability. Code is the source of truth.

4 Multi-Stage Delivery

Promote applications through multiple environments.



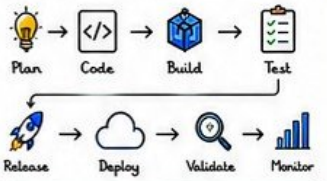
Key Practices

- ☑ Environment parity
- ☑ Automated gates
- ☑ Smoke & regression tests
- ☑ Approval & compliance
- ☑ Rollback strategy

Promote with confidence. Quality gates protect production.

5 Enterprise Release Workflow

End-to-end enterprise release process.



Key Practices

- ☑ Release calendar
- ☑ Change advisory board
- ☑ Automation & approvals
- ☑ Monitoring & alerts
- ☑ Post-release validation

Great releases are planned, validated, and observed.

6 Common Production Failures

What breaks production the most.

- ✗ Broken pipeline or YAML
- ✗ Failed deployment / bad config
- ✗ Missing environment variables
- ✗ Database migration failure
- ✗ Service dependency unavailable
- ✗ Resource limits / throttling
- ✗ Slow rollbacks
- ✗ Insufficient testing
- ✗ Permission / RBAC issues
- ✗ Secrets or credentials expired

Most failures are preventable. Prepare. Test. Validate. Monitor.

7 Senior Engineer Lessons

Lessons learned the hard way.

1. Always test in prod-like environments
2. Automate everything you can
3. Observe before you deploy
- ✓ Monitor what matters
- ✓ Keep rollbacks fast and simple
- ✓ Communication is critical
- ✓ Blameless postmortems drive growth
- ✓ Documentation saves incidents

Tools help. Judgment saves. Experience prevents repeat incidents.

8 Best Practices Summary

Principles for reliable delivery.

- Security by default
- Automate and standardize
- Observe and measure
- Collaborate and communicate
- Continuously improve
- Least privilege access

Do it right. Do it consistently. Deliver value safely.

Production Checklist

- ☑ Code reviewed and merged
- ☑ Build and tests pass
- ☑ Security scans clean
- ☑ Infrastructure validated
- ☑ Approvals completed
- ☑ Deployment succeeds
- ☑ Health checks pass
- ☑ Monitoring and alerts active
- ☑ Documentation updated
- ☑ Post-release review done

Sample Enterprise Release Flow

Remember

- ☑ You can't automate chaos.
- ☑ You can automate consistency.
- ☑ You can't remove risk.
- ☑ You can manage it.

Every incident teaches. Every lesson strengthens. Every release is a chance to improve.

Build Smart. Release Safely. Operate Reliably. Improve Continuously.

VERIQTA

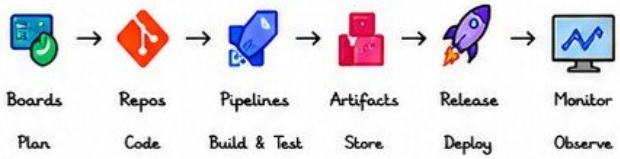
20. Volume 1 Production Review

Review. Reinforce. Remember. Apply.

Azure DevOps Production Handbook
Volume 1
Page 20 of 20

You've learned the building blocks.
Now review the big picture.
Know the workflow. Use the checklists.
Avoid mistakes. Think like a senior engineer.
Build Securely. Automate Reliably. Deliver Confidently.

1 Azure DevOps Workflow



Plan → Code → Build → Test → Package → Deploy → Monitor → Improve

End-to-end visibility. Full traceability. Continuous improvement.

2 YAML Pipeline Cheat Sheet

```yaml
trigger:
  - main
pool:
  vmImage: 'ubuntu-latest'
variables:
  buildConfiguration: 'Release'
steps:
  - checkout: self
  - task: UseDotNet@2
    inputs:
      packageType: 'sdk'
      version: '8.x'
  - script: dotnet restore
    displayName: 'Restore'
  - script: dotnet build --configuration $(buildConfiguration)
    displayName: 'Build'
  - script: dotnet test --configuration $(buildConfiguration)
    displayName: 'Test'
  - task: PublishBuildArtifacts@1
    inputs:
      PathtoPublish: '$(Build.ArtifactStagingDirectory)'
      ArtifactName: 'drop'
```

Key Points

- ☑ trigger: defines when pipeline runs
- ☑ pool: agent pool or VM image
- ☑ steps: tasks or scripts executed in order
- ☑ variables: reusable values
- ☑ artifacts: share outputs
- ☑ Use templates to DRY your YAML

Simple. Powerful. Version controlled. Reusable.

3 CI/CD Checklist

- ☑ Code committed and pushed
- ☑ Build succeeds
- ☑ All automated tests pass
- ☑ Code quality gates pass
- ☑ Artifacts published
- ☑ Security scans pass
- ☑ Infrastructure validated
- ☑ Deployment to target environment
- ☑ Post-deployment tests pass
- ☑ Monitoring and alerts enabled
- ☑ Rollback strategy verified
- ☑ Release documented

Don't skip steps.
Automate everything you can.

4 Security Checklist

- ☑ RBAC roles reviewed
- ☑ Least privilege access
- ☑ Branch policies enforced
- ☑ Secrets stored in Key Vault
- ☑ Service connections secured
- ☑ YAML pipelines reviewed
- ☑ Dependencies scanned
- ☑ Artifacts signed & verified
- ☑ Audit logs enabled
- ☑ Compliance requirements met
- ☑ Security gates in pipeline
- ☑ Regular access reviews

Security is not a step.
It's a mindset and a habit.

5 Troubleshooting Checklist

- ☑ Check pipeline logs
- ☑ Check agent status
- ☑ Verify variables and secrets
- ☑ Check service connections
- ☑ Validate YAML syntax
- ☑ Check task versions
- ☑ Check permissions (RBAC)
- ☑ Inspect deployment logs
- ☑ Verify environment access
- ☑ Check network/firewall rules
- ☑ Check external dependencies
- ☑ Review recent changes

Stay calm. Investigate.
Follow the checklist.

6 Common Mistakes

- ✗ Hardcoding secrets
- ✗ Skipping code reviews
- ✗ No automated tests
- ✗ Manual deployments
- ✗ No rollback plan
- ✗ Over-permissive access
- ✗ Ignoring pipeline failures
- ✗ No monitoring after deploy
- ✗ Large pipelines with no templates
- ✗ No documentation
- ✗ Not testing production-like
- ✗ Ignoring security scans

Small mistakes.
Big outages.

7 Senior Engineer Insights

- Think in systems, not tasks.
- Design for failure.
- Automate for consistency.
- Observability is your superpower.
- Security and reliability first.
- Document decisions and trade-offs.
- Measure everything.
- Continuously improve.
- Teach and share knowledge.
- Ownership drives excellence.

It's not about doing more.
It's about doing the right things.

8 Production Readiness Review

Environment Readiness

- ☑ Infrastructure in place
- ☑ Secrets and configs validated
- ☑ Monitoring and alerts configured
- ☑ Backups and DR validated

Application Readiness

- ☑ Build and tests stable
- ☑ Performance tested
- ☑ Security validated
- ☑ Business acceptance completed

Operational Readiness

- ☑ Runbooks documented
- ☑ Support and on-call ready
- ☑ Rollback tested and verified
- ☑ Communication plan ready

GO / NO-GO

- ☑ Meets requirements
- ☑ Secure by design
- ☑ Tested thoroughly
- ☑ Monitored & alerting
- ☑ Rollback ready
- ☑ Team aligned

READY?

No guesswork. Validate. Verify. Then ship.

9 Volume 1 Key Takeaways

- ✓ You now understand Azure DevOps end-to-end.
- ✓ You can build secure and reliable pipelines.
- ✓ You can deploy with confidence.
- ✓ You can troubleshoot like a pro.
- ✓ You think like a senior engineer.

Continue Your Journey

Keep building. Keep learning.
Practice. Automate. Improve.
The best engineers never stop learning and never stop shipping!

Great engineers build. Senior engineers design. Staff engineers influence. Principal engineers lead.

Build Smart. Automate Reliably. Deliver Confidently. Improve Continuously.